JWT-аутентификация

Access token, Bearer scheme и защита Express endpoints

Результаты занятия

- Объяснять назначение JSON Web Token
- Разбирать JWT на Header, Payload и Signature
- Понимать, что подписанный JWT не является зашифрованным
- Использовать стандартные claims sub, iat и exp
- Создавать access token после успешного login
- Передавать JWT через Authorization: Bearer
- Проверять JWT через jwt.verify()
- Создавать requireAuth middleware
- Защищать REST endpoints
- Корректно обрабатывать отсутствующий, неверный и просроченный token

Учебные вопросы

1	**Что такое JWT**	Структура, claims и цифровая подпись
2	**Создание access token**	jwt.sign(), secret, exp, sub, issuer и audience
3	**Bearer token**	Передача access token в HTTP Authorization header
4	**Проверка JWT**	jwt.verify(), requireAuth и req.user
5	**Безопасность JWT**	Срок жизни, секреты, payload и типичные ошибки

Учебный вопрос 1. Что такое JWT

JWT переносит claims и защищает их целостность подписью

JSON Web Token

- JWT — компактный формат представления claims, который может быть подписан для проверки подлинности и целостности.
- JWT часто используется как access token.
- После login сервер может выдать token клиенту.
- Клиент предъявляет token при обращении к protected endpoint.
- Сервер проверяет подпись и срок действия.
- Успешная проверка позволяет восстановить identity пользователя.

Claims — утверждения (данные) внутри JWT-токена, например идентификатор пользователя (sub), его роль (role) и срок действия токена (exp).

JWT состоит из трёх частей

- **Header** описывает тип token и алгоритм подписи.
- **Payload** содержит claims.
- **Signature** защищает Header и Payload от незаметного изменения.
- Три Base64URL-части разделены точками.
- Формат: header.payload.signature.

Base64URL — способ кодирования двоичных данных в текст, безопасный для передачи в URL; используется в JWT и не является шифрованием.

Как выглядит JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiI1Iiwicm9sZSI6InVzZXIiLCJpYXQiOjE3MDAwMDAwMDAsImV4cCI6MTcwMDAwMDkwMH0.
cu5kmkI9F5vlzlPINvH79dPuTl24MNR5fWIalfy5LME
```

JWT = Header + Payload + Signature

Header

Header: тип токена и алгоритм подписи

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload

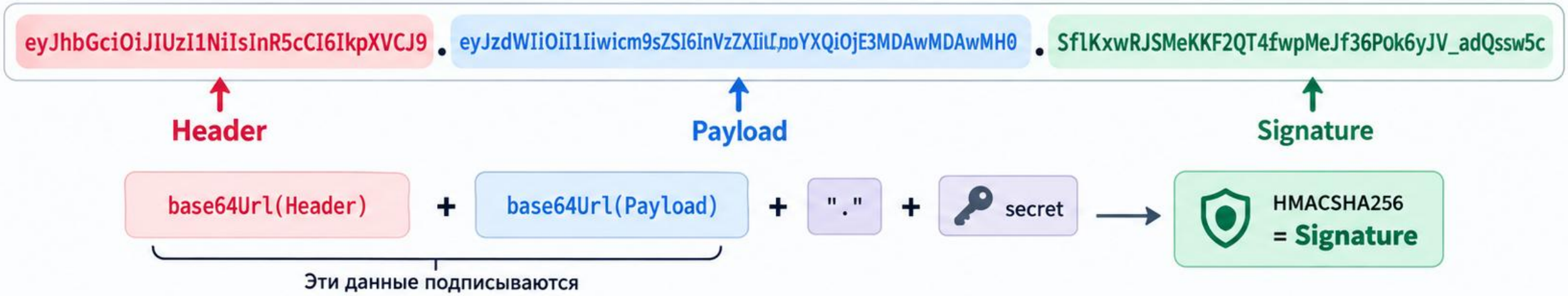
Payload: claims пользователя и метаданные

```
{
    "sub": "5",
    "role": "user",
    "iat": 1700000000,
    "exp": 1700000900
}
```

Signature

Signature: защищает от незаметного изменения

```
HMACSHA256(
    base64Url(Header) + "." +
    base64Url(Payload),
    secret
)
```



Payload читается и не является зашифрованным

Header, Payload и Signature

Часть	Что содержит	Секретна?
Header	alg, typ и другие параметры JOSE	Нет
Payload	Claims пользователя и token metadata	Нет
Signature	Результат криптографической подписи/MAC	Нет как отдельная часть, но создаётся с использованием секретного ключа

JWT не шифрует payload

- Base64URL encoding не является encryption.
- Header и Payload может прочитать любой, у кого есть token.
- Signature предотвращает незаметное изменение подписанного token.
- Не помещайте в Payload password, passwordHash, JWT_SECRET или другие секреты.
- JWT должен содержать минимально необходимые claims.

Полезные claims

Claim	Смысл	Пример
sub	Subject — идентификатор пользователя	5
iat	Issued At — время выпуска	Unix timestamp
exp	Expiration Time — срок действия	Unix timestamp
iss	Issuer — кто выпустил token	express-shop
aud	Audience — для кого предназначен token	express-shop-api
role	Private claim для роли	user

Claim exp обязателен для практического access token

- Access token не должен действовать бесконечно.
- exp задаёт момент, после которого token нельзя принимать.
- Библиотека jsonwebtoken проверяет exp при jwt.verify().
- Для учебного проекта используем короткий срок жизни 15 минут.
- Позже можно добавить refresh token отдельно.

Учебный вопрос 2. Создание access token

Token выдаётся только после успешной проверки credentials

Поток login → access token

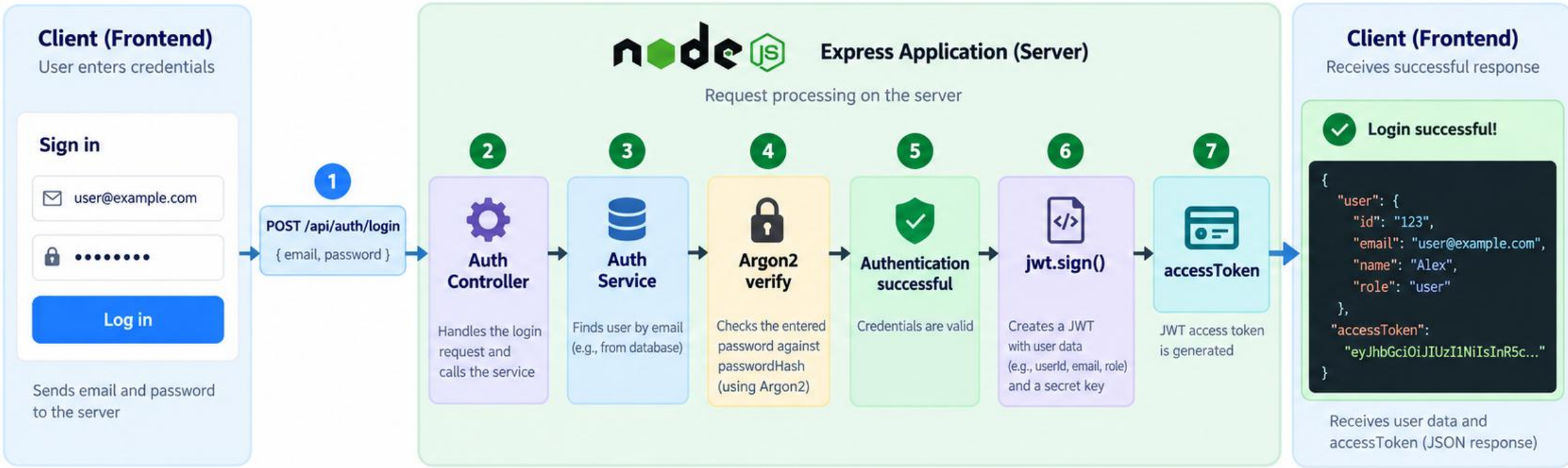
- Frontend отправляет email и password.
- Auth Service находит User.
- Argon2 проверяет password относительно passwordHash.
- Только после успешной Authentication создаётся JWT.
- JWT возвращается клиенту вместе с безопасными данными User.

Алгоритм выпуска JWT

1. **Проверить credentials** — authenticateUser() подтверждает email и password.
2. **Сформировать claims** — Использовать минимальные данные: sub и role.
3. **Подписать token** — jwt.sign() использует JWT_SECRET.
4. **Задать срок жизни** — expiresIn ограничивает время действия.
5. **Вернуть accessToken** — Controller отправляет token клиенту.

Login → credentials verified → JWT access token

Flow of JWT authentication in an Express application



Important: JWT is created **only after successful verification** of the user's credentials

If the email or password is incorrect, no token is generated, and the server returns an error (e.g., 401 Unauthorized).

Установка jsonwebtoken

```bash
npm install jsonwebtoken
```

JWT_SECRET

- JWT_SECRET — криптографический секрет для HMAC-подписи access token.
- Секрет хранится на сервере, а не во frontend.
- Не записывайте его прямо в исходный код.
- Храните его в .env или secrets manager.
- Для HS256 используйте криптографически случайный секрет не короче 256 бит.
- Не используйте обычный человекочитаемый password как JWT secret.

Переменные окружения

```
JWT_SECRET=<случайный секрет не менее 256 бит>
JWT_EXPIRES_IN=15m
JWT_ISSUER=express-shop
JWT_AUDIENCE=express-shop-api
```

tokenService.js: создание token

```javascript
import jwt from "jsonwebtoken";

export function createAccessToken(user) {
  return jwt.sign(
    { role: user.role },
    process.env.JWT_SECRET,
    {
      algorithm: "HS256",
      subject: String(user.id),
      issuer: process.env.JWT_ISSUER,
      audience: process.env.JWT_AUDIENCE,
      expiresIn: process.env.JWT_EXPIRES_IN
    }
  );
}
```

Что важно

- sub создаётся через option subject.
- role помещается в Payload как private claim.
- HS256 использует общий секрет.
- expiresIn создаёт exp.
- issuer и audience ограничивают область применения token.

Почему userId лучше хранить в sub

- sub — стандартный JWT claim для subject.
- Он естественно соответствует identity пользователя.
- Не нужно придумывать отдельное поле userId без необходимости.
- После verify значение decoded.sub можно преобразовать в Number.
- Стандартные claims делают назначение token понятнее.

Login Controller возвращает accessToken

```javascript
export async function login(req, res, next) {
  try {
    const { email, password } = req.body;
    const user = await authenticateUser(email, password);

    if (!user) {
      return res.status(401).json({
        error: "Invalid email or password"
      });
    }

    const accessToken = createAccessToken(user);

    res.json({ user, accessToken });
  } catch (error) {
    next(error);
  }
}
```

Что важно

- JWT создаётся только после успешной Authentication.
- passwordHash не попадает в token.
- Клиент получает accessToken для следующих запросов.

Успешный login с JWT

Запрос: POST /api/auth/login

Код • JSON

```
{
  "email": "alex@example.com",
  "password": "ExamplePassword123!"
}
```

Результат

```
{
  "user": {
    "id": 5,
    "name": "Alex",
    "email": "alex@example.com",
    "role": "user"
  },
  "accessToken": "eyJ..."
}
```

Пояснение

- accessToken используется для обращения к protected endpoints.

Учебный вопрос 3. Bearer token

JWT передаётся серверу через Authorization header

Bearer authentication scheme

- Access token обычно передаётся в HTTP header Authorization.
- Формат: Authorization: Bearer <token>.
- Bearer означает: предъявитель token получает права, связанные с ним.
- Поэтому утечка token опасна до момента его expiration.
- Token нельзя передавать в URL query string.

HTTP-запрос с Bearer token

```HTTP
GET /api/auth/me
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Как клиент использует access token

1	**Получить token**	Login response содержит accessToken.
2	**Сформировать Authorization**	Добавить префикс Bearer.
3	**Отправить protected request**	Например GET /api/auth/me.
4	**Сервер проверяет token**	requireAuth вызывает verifyAccessToken().
5	**Получить resource**	При успехе запрос продолжается.

Публичный и защищённый endpoint

Public endpoint

- GET /api/products
- Не требует JWT
- Доступен гостю
- requireAuth отсутствует

Protected endpoint

- GET /api/auth/me
- Требует Bearer token
- Без token → 401
- Использует requireAuth

Authorization: Bearer JWT → protected endpoint

Клиент
(браузер / веб-приложение)



Access Token (JWT)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Получен после успешного входа в систему (login).

1 Клиент отправляет **HTTP-запрос** к защищённому **endpoint** с **JWT** в заголовке **Authorization.**

```
GET /api/profile HTTP/1.1
Host: example.com
Authorization: Bearer <JWT>
Accept: application/json
```

Сервер (Express)

node express

Защищённый маршрут (endpoint)
Например: GET /api/profile

requireAuth (middleware)
Проверяет JWT и решает, пускать ли дальше.

2 **Извлечь токен из заголовка Authorization**

```
Authorization: Bearer <JWT>
```

3 **JWT verify**
Проверить подпись, срок действия (exp), и другие данные (например, роль).

4 **Принять решение**
(доступ разрешён или запрещён)

Токен валиден ✓

Запрос передаётся к защищённому endpoint (контроллеру).

Токен отсутствует, невалидный или истёк

Protected endpoint
(контроллер)

```
app.get('/api/profile', ..)
// Доступен только после
// успешной проверки JWT
```

JSON response (200 OK)

```
{
  "id": 1,
  "name": "Иван",
  "email": "ivan@example.com"
}
```

401 Unauthorized
Запрос блокируется requireAuth.

```
{
  "error": "Authentication required",
  "message": "Invalid or expired token"
}
```

Успешный ответ (JSON)
Данные возвращаются клиенту.

Ошибка 401
Сообщение об ошибке возвращается клиенту.

Только валидный Bearer JWT позволяет пройти к защищённому endpoint.

Почему не использовать query parameter

- URL может попасть в browser history.
- URL часто попадает в access logs и proxy logs.
- URL может передаваться как Referer.
- Authorization header предназначен для credentials.
- Access token рассматривается как чувствительный bearer credential.

Учебный вопрос 4. Проверка JWT

Доверяем claims только после успешного jwt.verify()

decode() и verify() — не одно и то же

- JWT можно Base64URL-декодировать без знания secret.
- decode показывает Payload, но не подтверждает подлинность.
- Protected endpoint должен использовать verify.
- verify проверяет signature и стандартные ограничения token.
- Нельзя принимать authorization decisions на основании только decoded payload.

tokenService.js: verify

```javascript
export function verifyAccessToken(token) {
  return jwt.verify(
    token,
    process.env.JWT_SECRET,
    {
      algorithms: ["HS256"],
      issuer: process.env.JWT_ISSUER,
      audience: process.env.JWT_AUDIENCE
    }
  );
}
```

Что важно

- algorithms явно ограничивает разрешённый алгоритм.
- issuer должен совпасть с ожидаемым.
- audience должен совпасть с ожидаемым.
- exp проверяется библиотекой при verify.

Извлечение Bearer token

```javascript
function getBearerToken(req) {
  const header = req.headers.authorization;

  if (!header) return null;

  const [scheme, token] = header.split(" ");

  if (scheme !== "Bearer" || !token) {
    return null;
  }

  return token;
}
```

Что важно

- Сначала читается Authorization header.
- Ожидается ровно Bearer scheme.
- При отсутствии token возвращается null.
- Строка JWT дальше передаётся verifyAccessToken().

requireAuth middleware

```javascript
export function requireAuth(req, res, next) {
  const token = getBearerToken(req);

  if (!token) {
    return res.status(401).json({
      error: "Authentication required"
    });
  }

  try {
    const payload = verifyAccessToken(token);

    req.user = {
      id: Number(payload.sub),
      role: payload.role
    };

    next();
  } catch {
    return res.status(401).json({
      error: "Invalid or expired token"
    });
  }
}
```

Что важно

- Без token запрос завершается 401.
- Claims используются только после verify.
- req.user содержит identity для следующих middleware.
- Неверный и просроченный JWT не пропускаются.

Защита маршрута

```javascript
router.get("/me", requireAuth, getCurrentUser);
```

getCurrentUser

```javascript
export async function getCurrentUser(req, res, next) {
  try {
    const user = await prisma.user.findUnique({
      where: { id: req.user.id },
      select: {
        id: true,
        name: true,
        email: true,
        role: true
      }
    });

    if (!user) {
      return res.status(401).json({ error: "User not found" });
    }

    res.json({ user });
  } catch (error) {
    next(error);
  }
}
```

Что важно

- JWT подтверждает identity.
- Controller может загрузить актуальный User из PostgreSQL.
- passwordHash не выбирается.
- Так приложение не зависит от всех пользовательских данных внутри token.

requireAuth: Bearer token → verify → req.user → Controller

Проверка JWT в middleware и передача данных пользователя в защищённый контроллер

Client

Отправляет запрос с JWT в заголовке

```
GET /api/products
HTTP/1.1
Host: shop.example.com
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6
IkpXVCJ9...
```

Express.js (Middleware) requireAuth

```
import jwt from 'jsomoktoken';

export function requireAuth(
  req, res, next) {
   const authHeader = req.headers
     .authorization;
   if (!authHeader ||
      !authHeader.startsWith('Bearer ')) {
     return res.status(401).json({
       error: 'Token required'
     });
  }

  const token = authHeader.split(' ')[1];
 }
  // ... verification дальше
}
```

Verify JWT

jwt.verify()

```
try {
 const payload = jwt.verify(
   token,
   process.env.JWT_SECRET,
   {
     algorthmms: ['HS256'],
     issuer: 'internet-shop',
     audience: 'internet-shop-client'
   }
 );
  req.user = {
    id: payload.sub,
    role: payload.role
  };
  next();
} catch (err) {
  return res.status(401).json({
    error: 'Invalid or expired token'
 };
```

Далее в контроллер

req.user доступен

```
app.get('/api/products',
  requireAuth,
 async (req, res) => {
   // req.user получен из JWT
   const userId = req.user.id;
   const userRole = req.user.role;

   const products = await
   productService.getAll();

   res.json({
     products,
      user: {
        id: userId,
        role: userRole
      }
   });
};
```

Response (200)

Доступ разрешён

```
{
 "products": [
   { "id": 1, "name": "Laptop" },
   { "id": 2, "name": "Mouse" }
 ],
 "user": {
   "id": "123",
   "role": "user"
}
```

Что происходит?

1. Клиент отправляет запрос с заголовком Authorization: Bearer <token>
2. Middleware извлекает токен из заголовка
3. jwt.verify() проверяет подпись, срок действия (exp), issuer, audience
4. При успехе данные пользователя сохраняются в `req.user`
5. Запрос передаётся в контроллер
6. При ошибке возвращается `401 Unauthorized`

Проверяется:

- Корректность подписи (signature)
- Срок действия (exp)
- Алгоритм (alg)
- Issuer (iss)
- Audience (aud)

Ошибка проверки

Invalid / Expired Token

Неверный или просроченный токен

```
return res.status(401).json({
  error: 'Invalid or expired token'
});
```

Response (401)

Доступ запрещён

```
{
  "error": "Invalid or expired token"
}
```

Результат:

- При успешной проверке — req.user содержит данные из JWT
- Контроллер получает доступ к защищённому ресурсу
- При ошибке — возвращается 401

Типичные ответы Authentication middleware

Ситуация	HTTP status	Ответ
Authorization отсутствует	401	Authentication required
Scheme не Bearer	401	Authentication required
Signature неверна	401	Invalid or expired token
Token просрочен	401	Invalid or expired token
Token валиден	Продолжение запроса	next()

Учебный вопрос 5. Безопасность JWT

JWT удобен, но требует строгой проверки и управления жизненным циклом

JWT Security Checklist

Ключевые правила безопасного использования JWT в Internet Shop

Безопасный JWT =
правильные настройки +
минимум данных + короткое время жизни

1 Secret (секретный ключ)

Надёжное хранение и генерация

- Используйте длинный случайный ключ (минимум 256 бит)
- Храните только на сервере (переменная окружения .env)
- Не храните в коде и в репозитории
- Используйте разные ключи для разных окружений (dev / prod)
- Регулярно ротируйте ключ

Пример (.env):

```
JWT_SECRET=4f9e2c7a8b1d3e6f9c0a7d
 72e5b8c1f0a9d3e6c7b2a1f9e4d8c3b7
```

Не используйте:

- простые пароли ("123456", "secret")
- ключи из примеров
- один и тот же ключ в разных проектах

2 Algorithm (алгоритм)

Используйте надёжные алгоритмы

- Используйте HS256 (для учебного проекта) или RS256 (для продакшена)
- Явно указывайте допустимые алгоритмы при проверке
- Не используйте устаревшие и небезопасные алгоритмы (например, none, HS1)
- Защищайтесь от алгоритмических атак (algorithm confusion)

Пример:

```
jwt.verify(token, process.env.JWT_SECRET, {
  algorithms: ['HS256'],
  issuer: 'internet-shop',
  audience: 'internet-shop-client'
}));
```

Не используйте:

- none (отключает подпись)
- алгоритмы, которые не поддерживаются
- алгоритмы без явной проверки

3 Claims (данные в токене)

Минимум и только необходимое

- Храните только необходимые данные (например, user id и роль)
- Используйте стандартные claims: sub, iat, exp, iss, aud
- Не храните чувствительные данные (пароли, email, личные данные)
- Используйте понятные и согласованные имена полей
- При необходимости добавляйте собственные claims

Пример Payload (payload):

```
{
   "sub":  "123",
   "role": "user",
   "iat":  1710000000,
   "exp":  1710003600,
   "iss":  "internet-shop",
   "aud":  "internet-shop-client"
}
```

Не добавляйте:

- пароли и хеши
- email (если не требуется)
- лишние данные (адрес, телефон и т.п.)

4 Expiration (срок действия)

Короткое время жизни токена

- Используйте короткий срок жизни access token (например, 15 минут)
- Указывайте exp при создании токена
- Проверяйте expiration при verify()
- Для долгой сессии используйте refresh token
- Реализуйте механизм отзыва токенов (например, при выходе из аккаунта)

Пример:

```
const token = jwt.sign(payload,
  process.env.JWT_SECRET, {
    algorithm: 'HS256',
    expiresIn: '15m',
    issuer: 'internet-shop',
    audience: 'internet-shop-client"
}));
```

Не делайте:

- слишком длинный срок жизни
- бессрочные токены
- игнорирование проверки exp

Дополнительные рекомендации

- Передавайте JWT только через HTTPS
- Храните токен на клиенте в безопасном месте (HttpOnly cookie или memory)
- Не передавайте токен в URL (параметрах запроса)
- Используйте refresh token с отдельными настройками безопасности

Итог

Безопасный JWT в Internet Shop:

- Сильный секретный ключ
- Надёжный алгоритм (HS256 / RS256)
- Минимум данных в claims
- Короткий срок действия
- Правильная проверка и защита от атак

Internet Shop

Безопасная аутентификация —
надёжная защита ваших данных

SECURITY • SIMPLICITY • TRUST

Минимальные правила JWT

- ✓ Использовать HTTPS
- ✓ Хранить JWT_SECRET только на сервере
- ✓ Генерировать сильный случайный secret
- ✓ Для HS256 использовать secret не короче 256 бит
- ✓ Явно ограничивать algorithms при verify
- ✓ Использовать exp
- ✓ Проверять issuer и audience
- ✓ Не помещать secrets в Payload
- ✓ Не логировать полный access token
- ✓ Не передавать token через URL

JWT payload нельзя считать доверенным до verify

- Клиент полностью контролирует входящий HTTP request.
- Он может прислать произвольную строку, похожую на JWT.
- Даже корректно декодированный Payload не доказывает подлинность.
- Только успешная cryptographic verification делает claims пригодными для использования.
- После verify всё равно обрабатывайте claims как обычные входные данные ожидаемого типа.

Короткий access token снижает риск

- Украденный bearer token можно использовать до его expiration.
- Короткий срок жизни уменьшает окно злоупотребления.
- В учебном проекте используем 15 минут.
- Длительная пользовательская сессия обычно требует отдельного refresh/session механизма.
- Refresh token не реализуем в этой лабораторной, чтобы не смешивать задачи.

Logout с JWT сложнее, чем с серверной session

- Самопроверяемый JWT может оставаться валидным до exp.
- Простое удаление token на клиенте завершает локальный login, но не отзывает уже украденную копию.
- Для принудительного отзыва применяют короткий access-token lifetime, refresh-token rotation или server-side denylist/status.
- Полный lifecycle токенов зависит от архитектуры приложения.

Полный JWT flow Internet Shop

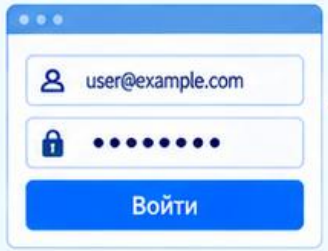
От входа в систему до доступа к защищённым API

Internet Shop
Безопасная аутентификация для вашего магазина

1. Login — аутентификация и получение JWT

Пользователь вводит логин и пароль. После успешной проверки создаётся JWT access token.

1 Client (Browser)
Вводит учётные данные



2 POST /api/auth/login
Отправка данных на сервер

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "P@ssw0rd!"
}
```

3 Auth Controller
Принимает запрос

```
router.post(
  '/auth/login',
  authController.login
);
```

4 Auth Service
Проверяет пароль

1. Находит пользователя в базе данных
2. Проверяет пароль через Argon2

```
await argon2.verify(
  user.passwordHash,
  password
);
```

Пароль верный

5 JWT Service
Создаёт токен

```
const token = jwt.sign(
  {
    sub: user.id,
    role: user.role
  },
  process.env.JWT_SECRET,
  {
    algorithm: 'HS256',
    expiresIn: '15m',
    issuer: 'internet-shop',
    audience: 'internet-shop-client"
  }
);
```

6 Response to Client
Возвращаем JWT

```
{
  "accessToken": "eyJhbGciOiJIUzI1...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "user": {
    "id": 123,
    "email": "user@example.com",
    "role": "user"
  }
}
```

2. Access Protected Endpoint — доступ к защищённым ресурсам

Клиент отправляет JWT в заголовке. Middleware проверяет токен и передаёт данные пользователя в контроллер.

1 Client (Browser)
Отправляет запрос с JWT

```
GET /api/products
Host: shop.example.com
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6...
```

2 Express.js
Маршрутизация

```
router.get(
  '/api/products',
  requireAuth,
  productController.getAll
);
```

3 requireAuth (Middleware)
Проверяет JWT

1. Извлекает токен из заголовка
2. Проверяет подпись (jwt.verify)
3. Проверяет срок действия (exp)
4. Проверяет issuer и audience
5. Создаёт req.user и вызывает next()

Токен действителен

```
req.user = {
  id: 123,
  role: 'user'
}
```

Неверный или просроченный токен

Response (401)
Доступ запрещён

```
{ "error": "Invalid or
  expired token" }
```

4 Product Controller
Обрабатывает запрос

```
app.get('/api/products',
  requireAuth,
  async (req, res) => {
    const products =
      await productService.getAll();
    res.json({ products });
  }
);
```

5 Response (200)
Данные защищённого ресурса

```
{
  "products": [
    { "id": 1, "name": "Laptop", "price": 999 },
    { "id": 2, "name": "Mouse", "price": 25 }
  ]
}
```

Что обеспечивает JWT?

- Безопасная аутентификация пользователей
- Передача минимально необходимых данных
- Защита от подделки (подпись)
- Ограниченное время действия
- Простая интеграция с REST API

Стандартные claims в JWT

- **sub** — идентификатор пользователя
- **iat** — время выдачи (issued at)
- **exp** — срок действия (expiration time)
- **iss** — издатель токена (issuer)
- **aud** — получатель токена (audience)
- **role** — роль пользователя (custom)

Используемые технологии

- Node.js + Express.js
- jsormolttokon (HS256)
- Argon2 (хеширование паролей)
- PostgreSQL (хранение пользователей)
- Middleware для проверки JWT
- HTTPS (обязателен в продакшене)

Итог

- Пользователь получает JWT после входа
- JWT передаётся в заголовке Authorization
- Каждый защищённый запрос проходит проверку
- При успехе контроллер получает req.user
- При ошибке возвращается 401
- Безопасный и надёжный доступ к API

JWT — простой, надёжный и современный способ аутентификации

Контрольные вопросы

1. Из каких трёх частей состоит JWT?
2. Почему Payload JWT нельзя считать секретным?
3. Для чего нужна Signature?
4. Что означает claim sub?
5. Для чего используется exp?
6. Где должен храниться JWT_SECRET?
7. Как передаётся Bearer token в HTTP?
8. Чем decode отличается от verify?
9. Что делает requireAuth middleware?
10. Почему access token желательно делать короткоживущим?

Итоги занятия

1. JWT — компактный формат claims, часто используемый как access token.
2. Подписанный JWT состоит из Header, Payload и Signature.
3. Payload кодируется, но не шифруется.
4. Access token выдаётся после успешной Authentication.
5. sub идентифицирует пользователя, exp ограничивает срок действия.
6. Клиент передаёт token как Authorization: Bearer <token>.
7. Protected routes используют requireAuth middleware.
8. jwt.verify() проверяет подпись и ограничения token.
9. JWT_SECRET должен быть сильным и храниться только на сервере.
10. Неверный или expired token приводит к 401.

Следующая лекция

- Авторизация и управление доступом
- Authentication vs Authorization на практике
- Role-Based Access Control
- user и admin
- requireRole middleware
- 403 Forbidden
- Защита административных endpoints